

Feature 4: tratamento de erros e persistência de dados

Para esta etapa crucial, seus projetos darão um salto qualitativo na construção de sistemas robustos e eficientes. Baseando-se na modelagem orientada a objetos da Feature 3, vocês aplicarão conceitos avançados da POO para criar arquiteturas flexíveis e escaláveis, garantindo que o software possa evoluir e lidar com a complexidade do mundo real. Além disso, introduzirão mecanismos essenciais para a resiliência da aplicação, como o tratamento de exceções, e a capacidade de interagir com o sistema de arquivos para persistir e recuperar dados, transformando seu projeto em uma aplicação completa e funcional.

1. Encapsulamento e modificadores de acesso:

Aprofundamento no encapsulamento: reforcem o uso de getters e setters, e demonstrem a proteção da integridade dos objetos, limitando o acesso direto aos atributos e controlando como eles são manipulados.

Modificadores de acesso: apliquem e justifiquem a utilização dos modificadores `public`, `private`, `protected` e `default` para atributos, métodos e classes, ilustrando seus impactos na visibilidade e acesso dentro e fora dos pacotes e hierarquias de herança.

Benefícios e aplicação prática: explorem como o encapsulamento e os modificadores de acesso contribuem para a segurança, manutenção e modularidade do código.

2. Herança:

Conceito: criem uma hierarquia de classes, utilizando a palavra-chave `extends` para estabelecer um relacionamento "é-um" (is-a) entre classes. Por exemplo, uma classe base que represente um conceito geral e classes derivadas que representem especializações desse conceito, como uma classe `Pessoa` e classes `Aluno` e `Professor` que a estendem.

Utilização da palavra-chave `super`: demonstrem o uso de `super` para invocar construtores da classe mãe e acessar membros da classe mãe (atributos e métodos) que foram sobrescritos ou ocultados.

Sobrescrita de métodos (`@Override`): implementem a sobrescrita de métodos em classes filhas para fornecer implementações específicas de comportamentos definidos na classe mãe. Utilizem a anotação `@Override` para garantir a correta intenção de sobrescrita e a clareza do código.

3. Polimorfismo:

Polimorfismo por subtipagem: ilustrem como objetos de diferentes classes podem ser tratados de forma uniforme através de uma referência de tipo comum (interface ou classe mãe). Demonstrem a capacidade de um objeto de assumir múltiplas formas em diferentes contextos, utilizando, por exemplo, uma lista que armazene diferentes tipos de objetos relacionados por herança ou interfaces.

Sobrecarga vs. Sobrescrita: apresentem exemplos claros que diferenciem a sobrecarga (múltiplos métodos com o mesmo nome, mas diferentes parâmetros na mesma classe) da sobrescrita (método com a mesma assinatura na classe filha, modificando o comportamento da classe mãe).

4. Abstração e interfaces:

Abstração: definam classes abstratas que não podem ser instanciadas diretamente e que contêm métodos abstratos (sem implementação), forçando as classes filhas a fornecerem suas próprias implementações. Isso é crucial para definir um contrato para comportamentos comuns sem detalhar a implementação.

Interfaces: criem interfaces para definir contratos de comportamento que classes podem implementar. Demonstrem o uso da palavra-chave `implements` e como as interfaces promovem flexibilidade e a capacidade de ter múltiplos comportamentos (herança múltipla de tipos), desacoplando a implementação da definição.

Palavra-chave `final`: apliquem `final` para variáveis (criando constantes), métodos (impedindo sobrescrita) e classes (impedindo herança), justificando seu uso para garantir imutabilidade, otimização ou design de segurança em pontos estratégicos do seu sistema.

5. Arrays e coleções:

Arrays: utilizem arrays para armazenar e manipular uma coleção de objetos do mesmo tipo. Demonstrem a declaração, inicialização, acesso a elementos e iteração (loop `for` tradicional e `for-each`), mostrando cenários onde o tamanho fixo é adequado.

Introdução a `ArrayList`: façam uso da classe `ArrayList` como uma coleção dinâmica para armazenar objetos. Apresentem suas vantagens em relação aos arrays de tamanho fixo, como adição, remoção, busca e redimensionamento automático de elementos, sendo mais flexível para listas de dados variáveis.

6. Tratamento de exceções:

Blocos `try-catch-finally`: implementem o tratamento de exceções usando os blocos `try`, `catch` e `finally` para lidar com erros em tempo de execução de forma controlada, evitando a interrupção abrupta do programa e fornecendo uma experiência mais robusta ao usuário.

Tipos de exceções: diferenciem exceções checadas (`checked`) de exceções não checadas (`unchecked`), explicando quando cada tipo deve ser tratado explicitamente (capturado ou declarado) ou pode ser propagado, respectivamente.

Lançando exceções (`throw`): demonstrem como lançar suas próprias exceções (`throw`) em situações onde uma condição de erro específica é detectada e precisa ser comunicada ao código chamador, permitindo uma resposta adequada a falhas.

Boas práticas: apliquem boas práticas no tratamento de exceções, como capturar exceções específicas (ao invés de genéricas), evitar "catch-all" vazios, e fornecer mensagens de erro informativas para depuração e feedback ao usuário.

7. Manipulação de arquivos:

Classes I/O: utilizem as classes `FileReader`, `FileWriter`, `BufferedReader` e `BufferedWriter` para interagir com o sistema de arquivos. Essas classes são fundamentais para operações de entrada e saída.

Leitura de arquivos de texto: implementem a funcionalidade de ler o conteúdo de um arquivo de texto, seja linha por linha ou caractere por caractere, processando os dados lidos para carregar informações ou configurações no seu sistema.

Escrita em arquivos de texto: implementem a funcionalidade de escrever dados (textos, resultados de processamento, estado do sistema etc.) em um arquivo de texto, incluindo a opção de sobrescrever ou anexar ao conteúdo existente, permitindo a persistência das informações do seu projeto.

Tratamento de exceções em I/O: apliquem o tratamento de exceções (`IOException`, `FileNotFoundException` etc.) de forma apropriada durante as operações de leitura e escrita de arquivos para lidar com cenários como arquivos não encontrados, problemas de permissão ou erros durante a gravação, garantindo a integridade dos dados e a estabilidade da aplicação.